Generative AI, RAG, LangChain & Agentic AI

Detailed Training Notes — Beginner Friendly • Architecture Focused • Future Reference

Goal: Understand how modern GenAI applications move from a user question to a grounded, useful response. These notes connect LangChain, LangGraph, LangSmith, RAG, chunking, embeddings, vector databases, retrieval optimization, memory, tools, and AI agents into one end-to-end picture.

Recommended Study Order

1. Introduction to Generative AI

Generative AI refers to AI systems that can generate new content such as text, code, images, audio, and structured data. A Large Language Model (LLM) learns statistical patterns from large amounts of text and predicts useful sequences of tokens.

Important limitation: An LLM is not automatically connected to your private documents or live business systems. Its built-in knowledge can be incomplete, outdated, or unsupported. RAG and tools are used to overcome these limitations.

Basic LLM Flow

User Prompt → Tokenization → Model Processing → Token Generation → Final Response

Key Terms

Term	Easy Meaning
Token	A small unit of text processed by the model.
Context Window	The maximum amount of information the model can consider at one time.
Prompt	Instructions or information sent to the model.
Inference	The process of generating an answer from a trained model.
Hallucination	A fluent but unsupported or incorrect model response.

2. System Prompt and User Prompt

System Prompt

The system prompt defines the model's overall behavior, role, boundaries, response format, and important rules. It has broader control over the application.

Example: “You are a financial document assistant. Answer only from retrieved documents. If evidence is missing, say that the information was not found.”

User Prompt

The user prompt is the current request from the user. Example: “What was the company's revenue in Q4?”

Why separate them?

The system prompt controls application behavior; the user prompt carries the task. Mixing everything into one uncontrolled prompt makes the application harder to maintain and less reliable.

3. What is LangChain?

LangChain is a framework for building applications around language models. It helps connect LLMs with prompts, documents, retrievers, vector stores, tools, output parsers, and other application components.

Simple idea: LangChain is the orchestration layer that helps you connect the pieces of an LLM application.

Common LangChain Components

• **Models:** Interfaces to chat models and embedding models.

• **Prompt Templates:** Reusable prompt structures with variables.

• **Document Loaders:** Load PDFs, web pages, text files, and other sources.

• **Text Splitters:** Divide large documents into chunks.

• **Embeddings:** Convert text into numerical vectors.

• **Vector Stores:** Store and search vectors.

• **Retrievers:** Return relevant documents or chunks.

• **Tools:** Functions or external systems an LLM/agent can call.

• **Output Parsers:** Convert model output into a required structure.

Conceptual Python Example

```
# Import a prompt template
from langchain_core.prompts import ChatPromptTemplate

# Create a reusable prompt with a variable called 'topic'
prompt = ChatPromptTemplate.from_template(
"Explain {topic} in simple beginner-friendly language."
)

# Fill the variable with an actual value
formatted_prompt = prompt.invoke({"topic": "RAG"})

# The result can then be passed to a compatible chat model
print(formatted_prompt)
```

Code explanation: The template is created once. The variable {topic} can change for every request. This avoids manually rebuilding the entire prompt.

4. What is LangGraph?

LangGraph is used to build stateful, multi-step, controllable workflows for LLM applications and agents. It is useful when an application needs branching, loops, retries, persistent state, human approval, or multiple agents.

Simple comparison: LangChain provides building blocks; LangGraph is useful for controlling a workflow made from those building blocks.

Example Workflow

User Query → **Classify Query** → **Retrieve Documents** → **Check Quality** → **Re-retrieve if needed** → **Generate Answer** → **Human Review if required**

Important Concepts

• **Node:** A step or function in the workflow.

• **Edge:** The connection that determines what step runs next.

• **State:** Shared information carried through the workflow.

• **Conditional Edge:** Chooses the next path based on a condition.

• **Checkpointing:** Saves workflow state so execution can resume.

5. What is LangSmith?

LangSmith is an observability, tracing, evaluation, and debugging platform for LLM applications. It helps developers understand what happened inside a chain, RAG pipeline, or agent.

Why it matters

• Trace prompts, model calls, retriever calls, and tool calls.

• Measure latency, token usage, failures, and application behavior.

• Compare versions of prompts or pipelines.

• Create datasets and evaluate application quality.

• Debug why a RAG system retrieved poor context or why an agent chose the wrong action.

6. LangChain Ecosystem

Component	Primary Purpose
LangChain	Build and connect LLM application components.
LangGraph	Create stateful, branching, cyclic, agentic workflows.
LangSmith	Trace, debug, evaluate, and monitor the application.

Memory shortcut: LangChain = Build. LangGraph = Orchestrate complex stateful workflows. LangSmith = Observe and evaluate.

7. What is RAG?

RAG stands for **Retrieval-Augmented Generation**. Before asking an LLM to answer, the system retrieves relevant information from an external knowledge source and places that information into the model's context.

Core Flow: User Question → Retrieve Relevant Knowledge → Add Knowledge to Prompt → LLM Generates Grounded Answer

Why RAG?

• Use private company or personal documents.

• Use information newer than the model's training knowledge.

• Reduce unsupported answers by grounding responses in evidence.

• Provide citations or source references.

• Update knowledge without retraining the entire LLM.

RAG is not model training

RAG usually does not change the LLM's internal parameters. It supplies relevant external context at query time. Fine-tuning changes model behavior or parameters; RAG supplies knowledge dynamically.

8. Types of RAG

Type	Description
Naive / Basic RAG	Retrieve top matching chunks and send them directly to the LLM.
Advanced RAG	Adds query rewriting, metadata filters, hybrid search, re-ranking, compression, and evaluation.
Modular RAG	Uses interchangeable modules and routing for different data sources or retrieval strategies.
Conversational RAG	Uses conversation context to rewrite or interpret follow-up questions.
Agentic RAG	An agent decides when, where, and how often to retrieve, possibly using multiple tools.
Graph RAG	Uses graph relationships/entities to support retrieval across connected information.

9. Offline Pipeline / Indexing Pipeline

The offline pipeline prepares documents so they can be searched efficiently later. It normally runs before the user asks a question and again when documents are added or updated.

Documents → Load → Clean → Split/Chunk → Create Embeddings → Store Vectors + Metadata in Vector DB

Step-by-Step

• **1. Ingestion:** Collect PDFs, Word files, web pages, database records, images/OCR text, etc.

• **2. Cleaning:** Remove irrelevant artifacts and normalize content.

• **3. Chunking:** Split large content into retrievable units.

• **4. Embedding:** Convert each chunk into a numerical vector.

• **5. Storage:** Save vector, original text, document ID, page number, language, date, and other metadata.

• **6. Indexing:** Prepare the vector store for efficient similarity search.

10. Online Pipeline / Response Pipeline

The online pipeline runs when a user asks a question.

User Query → Process Query → Decompose/Rewrite if Needed → Embed Query → Retrieve → Re-rank → Build Context → Construct Prompt → LLM → Parse/Validate → Response

Important Design Principle

Do not blindly send every retrieved chunk to the LLM. More context is not automatically better. Irrelevant context increases cost, latency, and confusion.

11. Chunking

Chunking means dividing a large document into smaller pieces that can be embedded, retrieved, and passed to the LLM.

Why is chunking required?

• Documents can exceed the model's context window.

• Embedding an entire large document creates a broad, less precise representation.

• Smaller chunks improve retrieval granularity.

• Good chunks preserve enough context to answer a question.

Chunk Size and Overlap

Chunk size controls how much content is stored in one unit. **Chunk overlap** repeats some content between neighboring chunks so important information is not lost at a boundary.

Trade-off: Too small → context is fragmented. Too large → retrieval becomes less precise and more expensive.

12. Types of Chunking

Type	How It Works	Best Use
Fixed-size	Split by a fixed number of characters or tokens.	Simple baseline.
Recursive	Try larger natural separators first, then smaller ones.	General documents.
Sentence-based	Split around sentence boundaries.	Readable text.
Semantic	Split when the meaning/topic changes.	Complex long-form content.
Structure-aware	Use headings, sections, tables, Markdown, HTML, or document structure.	Reports, manuals, policies.

Simple Chunking Example

```
# A very simple character-based chunking example

text = "This is a long document that we want to divide into smaller pieces."

chunk_size = 20 # Maximum characters in one chunk
overlap = 5 # Repeat 5 characters from the previous chunk

chunks = []

# Move through the text in steps of chunk_size - overlap
for start in range(0, len(text), chunk_size - overlap):
end = start + chunk_size
chunk = text[start:end]
chunks.append(chunk)

print(chunks)
```

Explanation: The overlap helps preserve information around chunk boundaries. In production, token-aware or structure-aware splitters are usually better than raw character splitting.

13. Vector Embeddings

An **embedding** is a numerical vector representing semantic information. Texts with similar meanings should generally be located closer together in vector space.

Example concept: “car” and “automobile” may have different words but similar vectors because their meanings are related.

Embedding Workflow

Text Chunk → Embedding Model → Vector such as [0.12, -0.44, 0.81, ...]

The query is embedded using a compatible embedding model. The system then searches for stored vectors that are most similar to the query vector.

14. Types of Embedding Models

Category	Description
General-purpose text embeddings	Designed for broad semantic search and retrieval.
Domain-specific embeddings	Optimized for areas such as medicine, finance, or scientific text.

Category	Description
Multilingual embeddings	Represent multiple languages in a shared or compatible vector space.
Multimodal embeddings	Represent more than one modality, such as text and images.
Sparse embeddings	Produce mostly-zero high-dimensional representations, often useful for lexical matching.
Dense embeddings	Produce compact continuous vectors commonly used in semantic search.

Important: Embedding quality should be tested on your own retrieval task. A popular model is not automatically the best model for your domain.

15. Vector Database

A **vector database** stores vectors and supports efficient similarity search. In RAG, it normally stores the embedding together with the original chunk and metadata.

Typical Stored Record

```
{
"id": "chunk_001",
"text": "The company's Q4 revenue was ...",
"embedding": [0.12, -0.44, 0.81, ...],
"metadata": {
"document": "annual_report.pdf",
"page": 42,
"year": 2025
}
}
```

Common Search Methods

• **Cosine Similarity:** Compares vector direction.

• **Dot Product:** Measures vector alignment; behavior depends on normalization.

• **Euclidean Distance:** Measures straight-line distance between vectors.

16. Types of Databases

Database Type	Main Purpose
Relational / SQL	Structured rows and tables with relationships.
Document Database	Flexible JSON-like documents.
Key-Value Store	Fast lookup using a key.
Graph Database	Entities and relationships.
Time-Series Database	Time-stamped measurements and events.
Vector Database	Similarity search over embeddings.
Hybrid Systems	Combine lexical, metadata, relational, and vector capabilities.

Do not assume a vector DB replaces every database. A production GenAI application may use SQL for transactions, object storage for original files, a vector index for semantic retrieval, and a graph database for relationships.

17. Query Processing

Query processing prepares the user's raw question before retrieval. This can include normalization, language detection, spelling handling, intent classification, conversation-aware rewriting, metadata extraction, and safety checks.

Example

User: “What about its revenue last year?”
Problem: “its” and “last year” depend on context.
Processed query: “What was Company X's revenue in 2025?”

18. Query Decomposition

Query decomposition breaks a complex question into smaller sub-questions.

Example: “Compare Company A and Company B's 2025 revenue and explain which grew faster.”

• Sub-query 1: What was Company A's 2025 revenue?

• Sub-query 2: What was Company A's previous-year revenue?

• Sub-query 3: What was Company B's 2025 revenue?

• Sub-query 4: What was Company B's previous-year revenue?

• Sub-query 5: Calculate and compare growth.

Why use it? A single retrieval query may not retrieve evidence for every part of a multi-part question. Decomposition improves coverage, but it also adds latency and complexity. Do not use it for every simple query.

19. Retrieval

Retrieval selects candidate chunks that may contain information needed to answer the query.

Common Retrieval Approaches

• **Dense Retrieval:** Semantic vector similarity.

• **Sparse/Lexical Retrieval:** Exact or term-based matching such as BM25-style retrieval.

• **Hybrid Retrieval:** Combines semantic and lexical search.

• **Metadata Filtering:** Restricts results by fields such as year, department, language, or document type.

• **Multi-query Retrieval:** Generates multiple search formulations.

20. Chunk Re-ranking

Initial retrieval is optimized for quickly finding candidates. **Re-ranking** performs a second, more precise ordering step so the most relevant chunks are placed first.

Pipeline: Retrieve Top 20 Candidates → Re-rank → Keep Best 5 → Send to LLM

Re-ranking Techniques

• **Cross-encoder re-ranking:** Scores the query and each candidate together.

• **LLM-based re-ranking:** Uses an LLM to judge relevance.

• **Rule/metadata-based scoring:** Adds business rules, freshness, authority, or document priority.

• **Reciprocal Rank Fusion:** Combines rankings from multiple retrieval systems.

Optimization benefit: Better evidence quality can reduce hallucination and reduce the number of chunks sent to the model.

21. Output Template and Output Parsing

An output template defines the expected response structure. An output parser converts or validates model output.

Example Requirement

```
# Desired structured response
{
"answer": "The answer to the user's question",
"sources": ["document_1", "document_2"],
"confidence": "high"
}
```

Structured output is useful when another application must consume the response. Validation is important because an LLM can still produce malformed or incomplete output.

22. Tools

A **tool** is an external capability that an LLM or agent can invoke. Tools allow the application to do things that cannot be solved reliably from model memory alone.

Examples

• Search a database.

• Call a weather or finance API.

• Use a calculator.

• Search internal documents.

• Send an email after authorization.

• Create a calendar event.

• Execute a business workflow.

Key difference: RAG retrieves knowledge. Tools can retrieve information *and/or perform actions*.

23. Memory

Memory allows a conversational application to preserve useful information across turns or workflow steps. However, simply storing the entire conversation forever is inefficient.

Types of Memory

Type	Meaning
Short-term / Conversation Memory	Recent turns needed for the current interaction.
Summary Memory	Compressed summary of older conversation.
Long-term Memory	Persistent information saved for future interactions.
Semantic Memory	Facts or knowledge retrieved by meaning.
Episodic Memory	Past events or interactions.
Procedural Memory	Instructions or learned workflows about how to perform tasks.
Agent State	Temporary working information used while an agent executes a workflow.

24. Memory Optimization

• Keep only relevant recent messages in the active context.

• Summarize older conversation instead of sending every token again.

• Store durable facts separately and retrieve them only when relevant.

• Use metadata and namespaces to isolate users, sessions, or projects.

• Apply expiration or deletion policies for temporary information.

• Do not store sensitive information unnecessarily.

• Evaluate whether retrieved memory actually improves answers.

Opportunity cost: More memory increases token usage and can inject stale or irrelevant information. Good memory systems are selective, not unlimited.

25. AI Agents / Agentic AI

An **AI agent** is a system in which a model can use available information, decide the next action, call tools, observe results, update state, and continue until it can produce a result or reaches a stopping condition.

Agent Loop

Goal → Inspect State → Decide Next Action → Call Tool → Observe Result → Update State → Continue or Answer

Agent vs Normal Chatbot

Normal Chatbot	Agentic System
Usually generates one direct response.	Can execute a multi-step workflow.
Limited action selection.	Chooses among tools/actions.
Often linear.	Can branch, loop, retry, or ask for approval.
Less stateful.	Can maintain explicit workflow state.

Important: Do not use an agent when a deterministic workflow is enough. Agents add flexibility, but also cost, latency, unpredictability, and evaluation difficulty.

26. Chatbot Optimization

Retrieval Optimization

• Improve document cleaning and chunking.

• Choose and evaluate an embedding model.

• Use metadata filters.

• Use hybrid retrieval when exact terms matter.

• Re-rank candidates.

• Remove duplicate or redundant context.

Prompt Optimization

• Give clear system instructions.

• Define how to handle missing evidence.

• Separate retrieved context from user instructions.

• Specify output format.

• Avoid excessively long prompts.

Performance Optimization

• Cache reusable results.

• Use smaller models for simple routing/classification tasks.

• Parallelize independent retrieval operations.

• Limit unnecessary tool calls.

• Control context size and output length.

Quality Optimization

• Build a representative evaluation dataset.

• Measure retrieval quality separately from generation quality.

• Trace failures.

• Use human review for high-risk workflows.

• Test adversarial and ambiguous queries.

27. How a Chatbot Processes a User Query

• 1. Receive user input.

• 2. Apply system-level rules and safety constraints.

• 3. Determine intent and whether retrieval/tools are required.

• 4. Resolve conversational references when needed.

• 5. Rewrite or decompose complex queries.

• 6. Retrieve candidate information.

• 7. Re-rank and filter evidence.

• 8. Build the final context and prompt.

• 9. Generate the response.

• 10. Parse, validate, cite, or format the output.

• 11. Update conversation state or memory when appropriate.

• 12. Trace and evaluate the run.

28. Why Query Decomposition is Used

Query decomposition improves retrieval for multi-part or reasoning-heavy questions. It increases recall because each sub-question can retrieve its own evidence.

But: Decomposition is not free. It can increase model calls, retrieval calls, latency, and the chance of combining inconsistent sub-results. Use it when query complexity justifies it.

29. How Re-ranking Optimizes Retrieval

Vector similarity alone may return semantically related but not directly useful chunks. Re-ranking applies a stronger relevance check to candidate results. This improves the quality of the final context while allowing the system to retrieve broadly first and narrow down later.

30. How Memory Optimizes Conversations

Memory prevents the user from repeating relevant context and supports continuity. Optimization means retrieving only the memory needed for the current request instead of repeatedly sending the full history.

31. How Tools Enhance LLM Capabilities

Tools connect the model to external data and actions. They solve the limitation that an LLM cannot inherently know live information or execute real-world operations. A well-designed tool system includes clear tool descriptions, strict inputs, permission checks, error handling, and stopping rules.

32. End-to-End Response Generation Pipeline

Complete Example:

• 1. User asks: “Compare the latest policies in these two company documents.”

• 2. System identifies this as a document-grounded comparison task.

• 3. Query processor extracts the two target companies and the policy topic.

• 4. Query decomposer creates focused sub-queries.

• 5. Retriever searches the vector database with metadata filters.

• 6. Re-ranker selects the strongest evidence.

• 7. Context builder removes duplicates and fits evidence within the token budget.

• 8. Prompt combines system instructions, user question, and retrieved evidence.

• 9. LLM generates the comparison.

• 10. Output parser enforces the required format.

• 11. Citations are attached to supporting sources.

• 12. LangSmith-style tracing/evaluation can be used to inspect the run.

• 13. Useful conversational state may be preserved for the next turn.

33. End-to-End RAG Architecture

OFFLINE / INDEXING
Documents → Loader → Cleaning → Chunking → Embedding Model → Vector Database + Metadata

ONLINE / RESPONSE
User → Query Processing → Query Decomposition/Rewrite → Query Embedding → Retrieval → Re-ranking → Context Construction → Prompt → LLM → Validation/Output → User

AGENTIC EXTENSION
User → Agent State → Decide → Retrieve or Call Tool → Observe → Retry/Branch if Needed → Generate Final Answer

34. Practical RAG Pseudocode

```
# 1. OFFLINE PIPELINE

documents = load_documents() # Read PDFs, text files, web pages, etc.
clean_documents = clean(documents) # Remove unwanted content
chunks = split_into_chunks(clean_documents)

for chunk in chunks:
vector = embedding_model.embed(chunk.text)

vector_db.add(
vector=vector, # Numerical meaning representation
text=chunk.text, # Original chunk text
metadata=chunk.metadata # Source, page, date, language, etc.
)


# 2. ONLINE PIPELINE

user_query = get_user_question()

processed_query = process_query(user_query)

query_vector = embedding_model.embed(processed_query)

candidates = vector_db.search(
query_vector,
top_k=20 # Retrieve a broader candidate set
)

ranked_chunks = rerank(
query=processed_query,
documents=candidates
)

best_context = ranked_chunks[:5] # Keep only strongest evidence
```

```
prompt = build_prompt(
system_instructions="Answer only from the supplied context.",
context=best_context,
user_question=user_query
)

answer = llm.generate(prompt)

return answer
```

Explanation: The offline pipeline prepares searchable knowledge. The online pipeline processes a live query, retrieves evidence, improves ranking, constructs context, and asks the LLM to generate the final answer.

35. Common Mistakes

• Using RAG without evaluating retrieval quality.

• Choosing chunk size randomly and never testing alternatives.

• Sending too many irrelevant chunks to the LLM.

• Assuming high vector similarity means the chunk answers the question.

• Using only dense retrieval when exact identifiers or keywords are important.

• Using agents for simple deterministic tasks.

• Saving unlimited conversation history as memory.

• Treating RAG as a guaranteed solution to hallucination.

• Mixing untrusted retrieved text with high-priority instructions without protection.

• Measuring only final answer quality and ignoring retrieval failures.

36. Interview Quick Revision

Q: What is LangChain?
A: A framework for connecting LLMs with prompts, retrievers, documents, tools, and other application components.

Q: What is LangGraph?
A: A framework for stateful, multi-step and agentic workflows with branching, loops, and persistent state.

Q: What is LangSmith?
A: A platform for tracing, debugging, evaluating, and monitoring LLM applications.

Q: What is RAG?
A: A technique that retrieves relevant external information and gives it to an LLM as context before generation.

Q: Why do we chunk documents?
A: To create smaller, retrievable units that fit model limits and improve retrieval precision.

Q: What is an embedding?
A: A numerical vector that represents semantic information.

Q: What does a vector DB do?
A: Stores vectors and performs similarity search to retrieve semantically related content.

Q: Why re-rank?
A: Initial retrieval finds candidates quickly; re-ranking more precisely orders them by relevance.

Q: Why query decomposition?
A: To break a complex question into smaller searchable sub-questions and improve evidence coverage.

Q: What is memory?
A: A mechanism for preserving useful conversational or workflow information across interactions.

Q: What is an AI agent?
A: A system that can decide actions, use tools, observe results, maintain state, and continue through multiple steps.

Q: RAG vs tool calling?
A: RAG mainly retrieves knowledge for grounding; tools can retrieve information and perform actions.

37. Final Mental Model

LangChain connects components.
LangGraph controls complex stateful workflows.
LangSmith helps trace and evaluate them.
RAG brings external knowledge.
Chunking creates retrievable units.
Embeddings represent semantic meaning numerically.
Vector databases search those representations.
Query processing and decomposition improve what is searched.
Re-ranking improves which evidence reaches the LLM.
Memory preserves selected useful context.
Tools connect the model to external capabilities.
Agents dynamically decide how to use these capabilities.

Best learning strategy: First understand the end-to-end pipeline. Then study each component independently. Finally, build one small RAG application and trace every step from document ingestion to final response. Memorizing definitions without understanding the data flow will not be enough for projects or interviews.